

Sistema de Marketing con Multiagentes

Agentes de IA · LangGraph · Claude · Streamlit

Ing. Julian Andres Quimbayo Castro — julian.quimbayo@corhuila.edu.co

Ing. Alvaro Hernan Alarcon Lopez — alvaro.alarcon@corhuila.edu.co

Ing. Jose Miguel Llanos Mosquera — jmllanosm@corhuila.edu.co

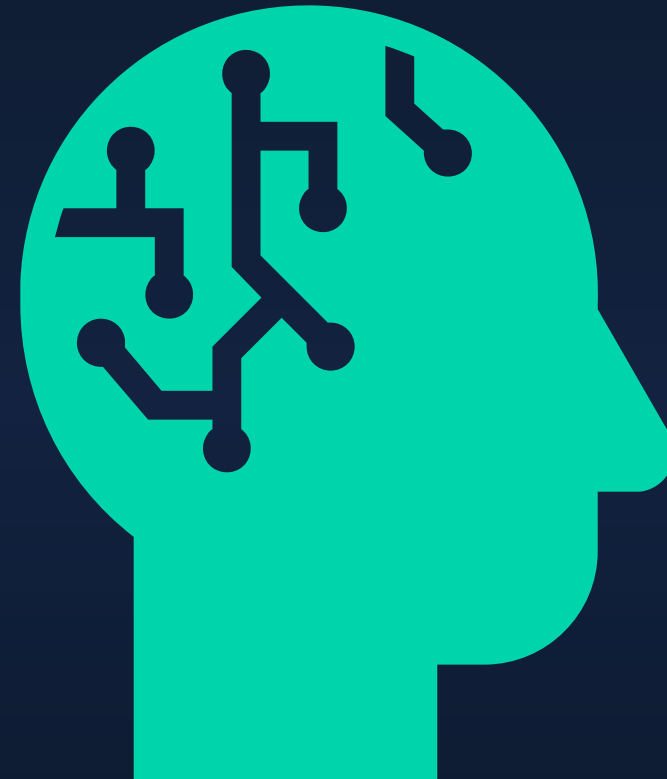
Corporación Universitaria del Huila — CORHUILA

Facultad de Ingeniería — Ingeniería de Sistemas

2026

Agenda

- 01 ¿Qué es un Agente de IA?
- 02 Anatomía de un Agente
- 03 Sistemas Multiagentes
- 04 Patrones de Arquitectura
- 05 LangGraph: Grafos de Estado
- 06 Componentes de LangGraph
- 07 Claude como Motor del Sistema
- 08 Buenas Prácticas con Claude
- 09 Streamlit como Frontend
- 10 Arquitectura del Proyecto
- 11 Agentes Especializados
- 12 Demo: Flujo de Trabajo
- 13 Skills del Proyecto
- 14 Conclusiones y Próximos Pasos



¿Qué es un Agente de IA?

Un agente de IA es un sistema de software autónomo que percibe su entorno, razona sobre él y ejecuta acciones para alcanzar objetivos específicos, utilizando un LLM como motor de razonamiento.

P

Percepción

Recibe inputs del entorno y usuario

R

Razonamiento

Analiza y decide el mejor curso

A

Acción

Ejecuta herramientas y genera outputs

L

Aprendizaje

Mejora con memoria y retroalimentación

Anatomía de un Agente



Sistemas Multiagentes

Múltiples agentes especializados que colaboran para resolver problemas complejos

Cada agente tiene:

- Un rol específico
- Herramientas propias
- Capacidad de comunicación
- Autonomía de decisión

Ventajas vs. Agente Único

Especialización por dominio



Escalabilidad horizontal



Tolerancia a fallos



Calidad superior en outputs



Flujos paralelos



Mantenimiento modular



Patrones de Arquitectura Multiagente

S

Supervisor

Un agente central coordina y delega tareas a agentes especializados

Usado en este proyecto

W

Swarm

Agentes autónomos se comunican entre sí de forma descentralizada

Sin punto central de fallo

J

Jerárquico

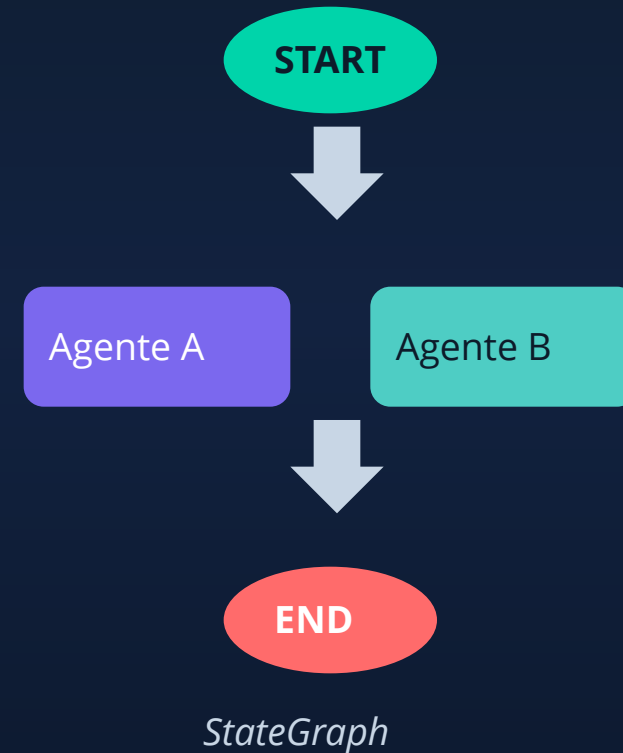
Múltiples niveles de supervisores y sub-equipos de agentes

Para sistemas complejos

LangGraph — Grafos de Estado para Agentes

Framework de LangChain para construir aplicaciones multiagente con estado, modeladas como grafos dirigidos.

- Los nodos son funciones (agentes o pasos)
- Las aristas definen transiciones condicionales
- El estado se comparte y persiste entre nodos
- Soporta checkpoints para reanudar flujos
- Integración nativa con LangChain y LLMs
- Ideal para workflows complejos con decisiones



Componentes de LangGraph

StateGraph

Grafo principal que define nodos, aristas y estado compartido. Es el contenedor de toda la lógica del agente.

Nodos (Nodes)

Funciones Python que procesan y transforman el estado. Cada agente es un nodo independiente.

Aristas (Edges)

Definen las transiciones entre nodos. Pueden ser condicionales basadas en el estado actual.

Checkpoints

Persisten el estado en cada paso. Permiten reanudar, depurar y crear puntos de recuperación.

Claude como Motor del Sistema

¿Por qué Claude?

Razonamiento
avanzado



Tool Use
nativo JSON



Contexto
200K-1M tokens



Respuestas
estructuradas



Multi-modelo
(Opus/Sonnet/Haiku
)



Seguridad
alineada



*En el proyecto: Claude como LLM principal
para los agentes de marketing vía API*

Pricing por Token (Abril 2026)

Modelo	Input / MTok	Output / MTok	Contexto
Opus 4.6	\$5.00	\$25.00	1M tokens
Sonnet 4.6	\$3.00	\$15.00	1M tokens
Haiku 4.5	\$1.00	\$5.00	200K tokens

● **Opus** → Tareas complejas, estrategia

● **Sonnet** → Balance calidad / coste

● **Haiku** → Alta velocidad, bajo coste

💡 Batch API: -50% • Prompt Caching: -90% en input

Buenas Prácticas con Claude

1 System Prompts Claros

Define el rol, contexto y restricciones del agente

2 Tool Use Estructurado

Usa JSON schemas para llamadas a herramientas

3 Prompt Chaining

Divide tareas complejas en pasos encadenados

4 Manejo de Errores

Implementa reintentos y fallbacks gracefully

5 Temperatura Baja

Usa temperature=0 para outputs deterministas

6 Evaluación Continua

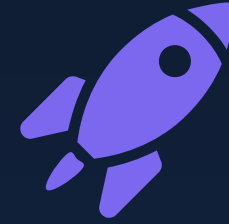
Mide calidad de outputs con métricas claras

Streamlit como Frontend



¿Por qué Streamlit?

- ▶ Prototipado rápido en Python
- ▶ Sin necesidad de HTML/CSS/JS
- ▶ Componentes interactivos nativos
- ▶ Chat UI con **st.chat_message**
- ▶ Despliegue fácil en la nube
- ▶ Ideal para demos de agentes IA



En el Proyecto

- ▶ Interfaz de chat con los agentes de marketing
- ▶ Visualización en tiempo real de respuestas
- ▶ Panel lateral con configuración
- ▶ Historial de conversaciones
- ▶ Feedback del usuario integrado
- ▶ Integración directa con **LangGraph**

Arquitectura del Proyecto

FRONTEND

Streamlit UI
Chat Interface · Config Panel · Visualización



ORQUESTACIÓN

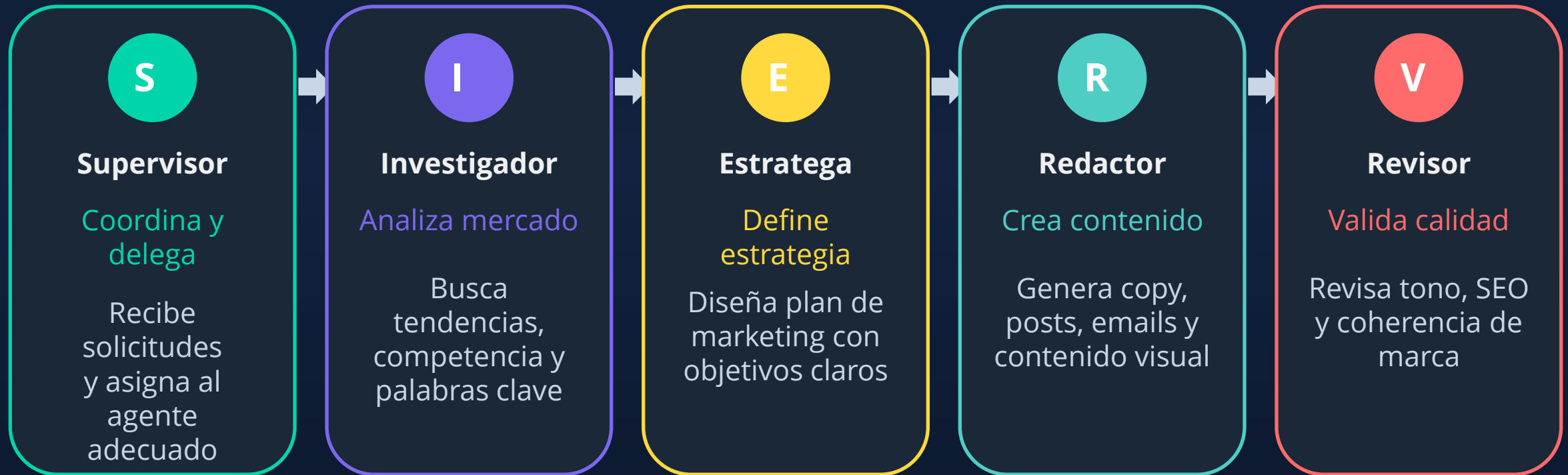
LangGraph StateGraph
Supervisor · Routing · State Management



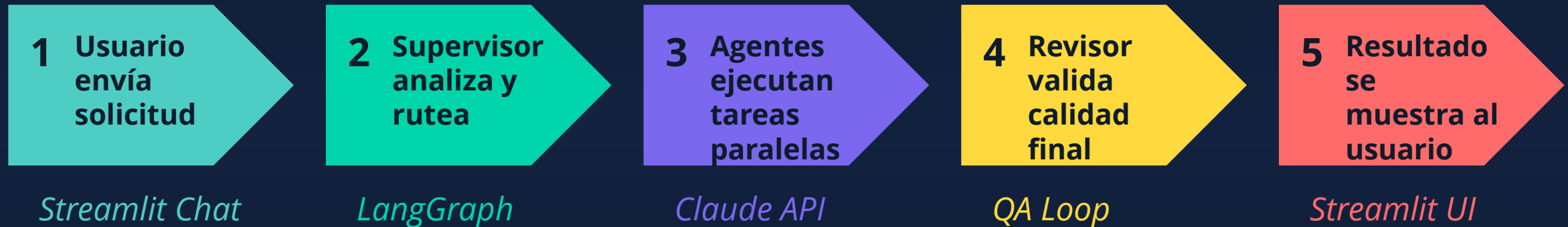
LLM ENGINE

Claude (Anthropic API)
Razonamiento · Tool Use · Generación de Contenido

Agentes Especializados del Sistema



Flujo de Trabajo: De Solicitud a Entregable



El Supervisor de LangGraph orquesta todo el flujo: analiza la solicitud, selecciona los agentes apropiados, gestiona la comunicación entre ellos, y consolida los resultados antes de entregarlos al usuario a través de Streamlit.

Demo

Sistema de Marketing con Multiagentes de IA

Skills Clave y su impacto en el proyecto

Python

Conecta todos los componentes del sistema

Sin él no hay agentes, ni grafos, ni UI

LangGraph

Convierte agentes aislados en un equipo

Controla el flujo, estado y decisiones

Claude

El motor que razona y genera estrategia

Transforma solicitudes en contenido real

Streamlit

Hace visible la IA para el usuario final

Chat en vivo y despliegue en minutos

Prompt Engineering

Define la personalidad de cada agente

Un mal prompt anula toda la arquitectura

Arquitectura

Permite crecer sin reescribir el sistema

Módulos independientes y escalables

Conclusiones y Próximos Pasos

LangGraph + Claude + Streamlit = Soluciones de marketing inteligentes, escalables y modulares



Software Libre y este Proyecto

01 100% Herramientas Open Source

Python, LangGraph, Streamlit y Claude API — todo el stack es gratuito, auditável y reproducible por cualquier persona o institución.

02 Sin vendor lock-in

Al usar software libre, el proyecto puede migrar, modificarse o mejorarse sin depender de licencias comerciales ni proveedores únicos.

03 Comunidad y colaboración

FLISol existe para difundir el software libre. Este proyecto es un ejemplo vivo: IA multiagente accesible, documentada y compatible.

04 Educación y soberanía tecnológica

Cualquier estudiante o desarrollador puede replicarlo, aprenderlo y extenderlo — sin barreras económicas ni restricciones de uso.

Gracias

Sistema de Marketing con Multiagentes de IA